

GPUMODE Lecture Study Notes: Lecture 68 – Landscape of GPU-Centric Communication

Core Problem the Lecture Addresses

The lecture addresses the widening imbalance between **GPU compute throughput** and **data movement capability**. Modern GPUs can perform floating-point and tensor-core operations at enormous rates, but memory bandwidth, network bandwidth, and latency have not improved at the same rate. As a result, communication becomes a dominant bottleneck in multi-GPU systems.

The central question is:

How can multi-GPU systems reduce CPU involvement in communication so that GPUs can initiate, manage, synchronize, and complete communication with minimal host-side overhead?

This motivates the idea of **GPU-centric communication**. In the lecture, GPU-centric communication is defined loosely as reducing the involvement of the CPU in the critical path of multi-GPU execution as much as possible.

The communication problem appears at two scales:

- **Intra-node communication:** communication among GPUs inside the same machine.
- **Inter-node communication:** communication among GPUs across different machines through a network interface card, or NIC.

A single GPU is no longer sufficient for frontier-scale high performance computing, deep learning training, or large-scale inference. The lecture uses examples from top supercomputers and large model training to show that thousands or tens of thousands of GPUs are common in serious systems. Therefore, it is not enough to optimize the kernel running on one GPU. One must also optimize the communication system surrounding it.

Required Background

From CPU-Centric to GPU-Centric Systems

Historically, the CPU controlled most computation and data movement. The GPU acted as an accelerator attached to the CPU. A typical execution flow looked like:

1. CPU prepares data.
2. CPU copies data to GPU.
3. CPU launches a GPU kernel.
4. GPU computes.
5. CPU copies results back.

In modern systems, this model is too restrictive. GPUs now perform most of the heavy computation. The CPU still handles operating system duties, process management, and some orchestration, but it is no longer the true center of computation.

The lecture frames the CPU increasingly as an **advanced memory controller and control processor**, while GPUs, NICs, and switches become more autonomous.

System Imbalance

The key performance trend is that compute capability grows faster than memory and network movement capability. A useful ratio is the **FLOP-to-byte ratio**:

$$\text{FLOP-to-byte ratio} = \frac{\text{Peak floating-point operations per second}}{\text{Bytes per second of data movement bandwidth}}.$$

If this ratio increases over time, then every byte moved must support more computation to keep the processor busy. This means algorithms must either increase data reuse or suffer from data movement bottlenecks.

For tensor-core GPUs, the lecture notes that peak tensor-core throughput has improved dramatically from earlier NVIDIA generations to Blackwell-era GPUs, while memory bandwidth has improved much more slowly. The implication is:

$$\text{Compute growth} \gg \text{Memory and network bandwidth growth}.$$

Therefore, communication optimization becomes more important with each GPU generation.

Communication as Data Movement

Communication is not only network communication. It includes any movement of data across memory hierarchy boundaries:

- global memory to GPU registers,
- GPU memory to CPU memory,
- one GPU memory to another GPU memory,
- one node to another node through a NIC,
- one GPU to another through NVLink, PCIe, InfiniBand, or another fabric.

The lecture emphasizes that communication performance depends on both:

- **data path:** where the bytes physically move;
- **control path:** who initiates, registers, triggers, and synchronizes the transfer.

Basic GPU Execution Terms

A CUDA program typically organizes work using a hierarchy:

grid $\rightarrow$ thread blocks $\rightarrow$ warps $\rightarrow$ threads.

Important concepts include:

- **SM:** streaming multiprocessor, the hardware unit that schedules and executes warps.
- **Warp:** a group of typically 32 threads executing in SIMT fashion.
- **Thread block:** a group of threads that can cooperate through shared memory and synchronization.
- **Global memory:** large device memory, high bandwidth but relatively high latency.
- **Shared memory:** programmer-managed memory local to an SM.
- **Registers:** fastest per-thread storage.
- **Occupancy:** how many warps or thread blocks can reside on an SM.

Communication libraries interact with this execution model differently. Some communication calls occur only at host-side kernel boundaries, while others can be issued directly from inside a running GPU kernel.

Main Concepts

GPU-Centric Communication

GPU-centric communication attempts to minimize the CPU's role in multi-GPU communication. The CPU may still initialize memory, launch kernels, or configure communication contexts, but the goal is to remove the CPU from the latency-sensitive critical path.

The lecture organizes communication mechanisms according to four questions:

1. **API:** Where is the communication API called, host or device?
2. **Data path:** Does the data pass through CPU memory, or does it move directly between devices?
3. **Register:** Who creates and registers the message with the NIC?
4. **Trigger:** Who rings the NIC doorbell and starts the transfer?

For intra-node communication, the main dimensions are API location and data path. For inter-node communication, NIC registration and triggering also matter.

Intra-Node Communication Categories

Intra-node communication refers to communication among GPUs in the same machine.

Host Native

In host-native communication, both the API call and the data path involve the CPU. If GPU 0 wants to communicate with GPU 1, the transfer may proceed as:

$$\text{GPU}_0 \rightarrow \text{CPU memory} \rightarrow \text{GPU}_1.$$

This requires two copies. The CPU is on the control path and the data path.

$$T_{\text{host-native}} \approx T_{\text{GPU}_0 \rightarrow \text{CPU}} + T_{\text{CPU} \rightarrow \text{GPU}_1} + T_{\text{host overhead}}.$$

This model is simple but inefficient for high-bandwidth GPU-to-GPU communication.

Host-Controlled Peer-to-Peer

With GPU peer-to-peer access, the CPU still calls the communication API, but data moves directly between GPUs:

$$\text{GPU}_0 \rightarrow \text{GPU}_1.$$

The data path avoids CPU staging. This became practical through technologies such as GPUDirect peer-to-peer access over PCIe and later NVLink-based systems.

The performance model becomes:

$$T_{\text{host-controlled}} \approx T_{\text{GPU}_0 \rightarrow \text{GPU}_1} + T_{\text{host launch/control}}.$$

Most common communication libraries, including CUDA-aware MPI and NCCL, have traditionally used host-side APIs. The host submits communication work, often into a CUDA stream, but the data itself can move directly among GPUs.

Device Native

In device-native communication, the API is called from GPU code, and the data path is also device-to-device. This allows a running kernel to issue communication without returning to the CPU.

A useful mental model is:

GPU kernel produces data $\rightarrow$ GPU issues communication $\rightarrow$ GPU continues computation.

This enables finer overlap between computation and communication. NVSHMEM is a key example because it exposes device-side communication APIs.

Host Fallback

In host-fallback communication, the programmer may write device-side communication code, but peer-to-peer access is unavailable or disabled. The implementation may fall back to staging through the host.

The important warning is that code may still run, but the performance path changes:

Expected path : $\text{GPU}_0 \rightarrow \text{GPU}_1$,

while the actual path may become:

Actual path : $\text{GPU}_0 \rightarrow \text{CPU memory} \rightarrow \text{GPU}_1$.

This can silently introduce much higher latency and lower bandwidth.

Inter-Node Communication Categories

Inter-node communication involves GPUs on different nodes. A NIC participates in the data transfer. The lecture adds two additional operations beyond API and data path:

- **Register:** who constructs and registers a message with the NIC;
- **Trigger:** who signals the NIC to begin the transfer.

Host Native Inter-Node Communication

In the most CPU-centric model, GPU data is copied to host memory, the CPU prepares the network message, and the NIC transfers from host memory to the remote node.

The path is roughly:

GPU memory $\rightarrow$ CPU memory $\rightarrow$ NIC $\rightarrow$ network $\rightarrow$ remote CPU memory $\rightarrow$ remote GPU memory.

This maximizes CPU involvement and copy count.

Pinned Host Native Communication

GPUDirect version 1.0 introduced shared pinned host memory between the GPU and NIC. The GPU can write into a pinned host region, and the NIC can read from it directly.

This removes one extra host-side copy:

GPU memory $\rightarrow$ pinned host memory $\rightarrow$ NIC $\rightarrow$ network.

Pinned host memory reduces latency and avoids some CPU-side memory movement, but the data still stages through host memory.

GPUDirect RDMA

GPUDirect RDMA allows the NIC to read from and write to GPU memory directly over PCIe. The GPU memory is exposed to the NIC through a BAR region.

The path becomes:

GPU memory $\leftrightarrow$ NIC $\leftrightarrow$ network $\leftrightarrow$ remote NIC $\leftrightarrow$ remote GPU memory.

This is a major milestone because it removes host staging from the data path.

A simplified latency model is:

$$T_{\text{RDMA}} \approx T_{\text{NIC reads GPU memory}} + T_{\text{network transfer}} + T_{\text{remote NIC writes GPU memory}}.$$

This mechanism is widely used by CUDA-aware MPI, NCCL, NVSHMEM, and related GPU-aware communication systems.

GPU-Triggered Networking

GPU-triggered networking improves the control path. The CPU may pre-register messages, but the GPU triggers the transfer by signaling the NIC. This is often associated with GPUDirect Async.

The CPU is still involved in setup, but the GPU can ring the NIC doorbell. Conceptually:

CPU pre-registers message, GPU produces data, GPU triggers NIC.

This is useful for intrakernel communication because a GPU kernel can produce data and trigger communication without returning to the CPU at every communication boundary.

Device Native Inter-Node Communication

In the most GPU-centric model, the GPU handles API-level communication, message construction, registration, and triggering. The CPU is mostly used only for initialization and kernel launch.

This is closest to **CPU-free communication**. It is supported by technologies such as InfiniBand GPUDirect Async variants and NVSHMEM configurations, though the lecture notes that this is not always enabled on production supercomputers.

The idealized path is:

GPU → NIC → network → remote NIC → remote GPU.

The control path is also GPU-driven:

GPU constructs message → GPU registers or references message → GPU triggers NIC.

Hardware-Level Explanation

PCIe, NVLink, and NVSwitch

PCIe

PCIe is the traditional host-device interconnect. It connects GPUs, NICs, and CPUs within a node. PCIe bandwidth is much lower than modern GPU memory bandwidth and lower than NVLink bandwidth.

If GPUs communicate through PCIe, performance is limited by PCIe topology and root complex placement. For GPUDirect RDMA, the GPU and NIC generally need to be reachable under a compatible PCIe topology.

NVLink

NVLink provides high-bandwidth, low-latency GPU-to-GPU communication. Compared with PCIe, it is designed for direct accelerator-to-accelerator traffic.

The lecture notes that each NVLink generation improves bandwidth substantially. A simplified comparison is:

$$B_{\text{NVLink}} \gg B_{\text{PCIe}},$$

where B denotes bandwidth.

NVLink is especially important for collective communication, tensor parallelism, pipeline parallelism, and fine-grained communication among GPUs inside a node.

NVSwitch

NVLink alone does not automatically provide scalable all-to-all routing across many GPUs. NVSwitch provides switching and routing among many GPUs, enabling high-bandwidth multi-GPU systems.

Without a switch, a message may need to route through intermediate GPUs. With NVSwitch, the system can provide more direct all-to-all connectivity.

For N GPUs, a full pairwise communication pattern contains:

$$\frac{N(N-1)}{2}$$

undirected GPU pairs. As N grows, direct connectivity becomes difficult without a switch fabric.

NICs and Smart NICs

A NIC is not merely a simple Ethernet adapter in modern GPU clusters. High-performance NICs such as NVIDIA ConnectX or BlueField-class devices can participate in RDMA, queue management, message triggering, offload, and sometimes computation or packet processing.

For GPU-centric communication, the NIC matters because it may:

- read directly from GPU memory;
- write directly into remote GPU memory;
- maintain queue pairs and work queues;
- receive doorbell notifications;
- execute offloaded communication protocol logic;
- participate in congestion control or routing.

The more work the NIC and GPU can do without CPU intervention, the more GPU-centric the communication becomes.

Memory Hierarchy and Communication

A GPU communication transfer often begins from or ends in global memory. However, the performance of the surrounding computation depends on the entire memory hierarchy:

registers $\leftrightarrow$ shared memory $\leftrightarrow$ L1 cache $\leftrightarrow$ L2 cache $\leftrightarrow$ HBM $\leftrightarrow$ interconnect or NIC.

If a kernel produces data in registers, that data generally must be stored to global memory before a communication library or remote GPU can read it. Some device-side communication models allow more precise timing: a thread block can produce a region of global memory and then signal that the data is ready without waiting for the entire kernel to finish.

SM-Level Overlap

Device-side communication enables a valuable pattern:

some thread blocks communicate while other thread blocks compute.

This can improve overlap if the application has fine-grained or irregular communication. However, it also introduces complexity:

- communication consumes GPU execution resources;

- polling or waiting can waste SM cycles;
- synchronization bugs become easier to introduce;
- profiling becomes harder because communication appears as memory access.

Visual Concept

Draw a node with multiple GPUs connected by NVLink or NVSwitch. Then draw a NIC connected to the GPUs through PCIe. Show one path where data goes through CPU memory, and another path where the NIC reads GPU memory directly. Label the second path as GPUDirect RDMA.

Mathematical Reasoning

Data Movement Cost Model

A basic communication time model is:

$$T_{\text{comm}} = \alpha + \frac{n}{B} + T_{\text{control}} + T_{\text{sync}},$$

where:

- α is startup latency;
- n is the number of bytes transferred;
- B is effective bandwidth;
- T_{control} is API, registration, launch, or CPU orchestration overhead;
- T_{sync} is synchronization cost.

GPU-centric communication attempts to reduce T_{control} and sometimes T_{sync} . Direct data paths increase effective B by avoiding unnecessary copies.

Copy Count Model

Let a transfer from GPU A to GPU B move n bytes. If the path stages through CPU memory, the system may perform two copies:

$$C_{\text{host-staged}} = 2n.$$

If peer-to-peer transfer is available:

$$C_{\text{peer}} = n.$$

The bandwidth-visible cost is roughly:

$$T_{\text{host-staged}} \approx \frac{n}{B_{\text{GPU} \rightarrow \text{CPU}}} + \frac{n}{B_{\text{CPU} \rightarrow \text{GPU}}},$$

while direct peer transfer is:

$$T_{\text{peer}} \approx \frac{n}{B_{\text{GPU} \rightarrow \text{GPU}}}.$$

If $B_{\text{GPU} \rightarrow \text{GPU}}$ is high, as with NVLink, direct peer transfer can be much faster.

Arithmetic Intensity and Communication Intensity

Arithmetic intensity is:

$$I_{\text{arith}} = \frac{\text{FLOPs}}{\text{Bytes moved}}.$$

A related distributed metric is communication intensity:

$$I_{\text{comm}} = \frac{\text{Bytes communicated}}{\text{FLOPs performed}}.$$

High I_{arith} is good for keeping GPU compute units busy. Low I_{comm} is good for distributed scalability. A communication-heavy algorithm has large I_{comm} , meaning it moves many bytes for each unit of useful computation.

The effective performance can be bounded by:

$$P_{\text{achievable}} \leq \min \left(P_{\text{peak}}, B_{\text{mem}} I_{\text{arith}}, \frac{B_{\text{net}}}{I_{\text{comm}}} \right),$$

where:

- P_{peak} is peak compute throughput;
- B_{mem} is memory bandwidth;
- B_{net} is network bandwidth;
- I_{arith} measures local data reuse;
- I_{comm} measures distributed communication cost.

Overlap Model

If computation and communication are serialized, total time is:

$$T_{\text{serial}} = T_{\text{compute}} + T_{\text{comm}}.$$

If they overlap perfectly:

$$T_{\text{overlap}} = \max(T_{\text{compute}}, T_{\text{comm}}).$$

The maximum possible speedup from overlap is:

$$S_{\text{overlap}} = \frac{T_{\text{compute}} + T_{\text{comm}}}{\max(T_{\text{compute}}, T_{\text{comm}})}.$$

Device-side communication can improve overlap because communication can begin before a full kernel completes. However, perfect overlap is rare because communication may contend for memory bandwidth, SM cycles, interconnect bandwidth, or NIC resources.

Collective Communication Cost

For a ring all-reduce over p GPUs with message size n , the approximate bandwidth cost per GPU is:

$$V_{\text{ring all-reduce}} = 2 \frac{p-1}{p} n.$$

A simplified latency-bandwidth model is:

$$T_{\text{ring all-reduce}} \approx 2(p-1)\alpha + 2 \frac{p-1}{p} \frac{n}{B}.$$

This explains why NCCL and RCCL optimize collective algorithms based on topology and message size. For small messages, latency dominates. For large messages, bandwidth dominates.

Topology-Aware Communication

The effective bandwidth between GPU i and GPU j depends on topology:

$$B_{ij} = f(\text{PCIe}, \text{NVLink}, \text{NVSwitch}, \text{NIC locality}, \text{routing}).$$

A communication library should choose algorithms that minimize traffic over weak links. For example, if intra-node NVLink bandwidth is much higher than

inter-node InfiniBand bandwidth, the algorithm should aggregate or reduce locally before sending across nodes when possible.

Code Walkthrough

CUDA-Aware MPI

CUDA-aware MPI allows MPI calls to accept GPU buffers. The MPI implementation determines whether a pointer refers to host memory or device memory.

A simplified example is:

```
#include <mpi.h>
#include <cuda_runtime.h>

void exchange_with_mpi(float *d_send, float *d_recv, int count,
                      int peer, cudaStream_t stream) {
    MPI_Request reqs[2];

    cudaStreamSynchronize(stream);

    MPI_Irecv(d_recv, count, MPI_FLOAT, peer, 0,
             MPI_COMM_WORLD, &reqs[0]);

    MPI_Isend(d_send, count, MPI_FLOAT, peer, 0,
             MPI_COMM_WORLD, &reqs[1]);

    MPI_Waitall(2, reqs, MPI_STATUSES_IGNORE);
}
```

The important detail is the explicit call to `cudaStreamSynchronize`. MPI is traditionally not CUDA stream aware. If a CUDA kernel writes `d_send` in a stream, MPI may not know that the stream still contains pending work. The programmer must ensure that the data is ready before MPI reads it.

Hardware interpretation:

- `d_send` and `d_recv` are device pointers.
- CUDA-aware MPI can detect that these are GPU buffers.
- For off-node communication, GPUDirect RDMA may allow the NIC to read directly from `d_send`.
- For intra-node communication, CUDA IPC or peer-to-peer access may be used.
- Without GPU-aware support, the implementation may stage through host memory.

Potential bottlenecks include:

- stream synchronization serializes computation and communication;
- small messages may suffer from high latency;
- performance depends on MPI implementation, UCX configuration, and topology.

NCCL Send and Receive

NCCL is stream aware. Communication operations are inserted into CUDA streams.

```
#include <nccl.h>
#include <cuda_runtime.h>

void exchange_with_nccl(float *d_send, float *d_recv, int count,
                        int peer, ncclComm_t comm,
                        cudaStream_t stream) {
    ncclGroupStart();

    ncclRecv(d_recv, count, ncclFloat, peer, comm, stream);
    ncclSend(d_send, count, ncclFloat, peer, comm, stream);

    ncclGroupEnd();
}
```

The `ncclGroupStart` and `ncclGroupEnd` calls allow NCCL to group communication operations. NCCL can then schedule them more efficiently, potentially reducing congestion or avoiding deadlock patterns.

Hardware interpretation:

- The communication is enqueued into a CUDA stream.
- Operations in the same stream respect stream order.
- If a previous kernel in the stream produced `d_send`, NCCL waits according to stream semantics.
- NCCL chooses algorithms and transports based on topology, message size, and configuration.

Compared with MPI, NCCL integrates more naturally with CUDA streams. This makes overlap with GPU computation easier to express.

NVSHMEM Device-Side Put with Signal

NVSHMEM exposes a partitioned global address space model. Each processing element, or PE, has private memory and symmetric memory. Other PEs can

access symmetric memory using one-sided operations.

A representative device-side example is:

```
#include <nvshmem.h>
#include <nvshmemx.h>

__global__ void producer_kernel(float *remote_buffer,
                                uint64_t *remote_signal,
                                const float *local_data,
                                int n,
                                int peer_pe) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;

    if (tid < n) {
        float value = local_data[tid] * 2.0f;
        remote_buffer[tid] = value;
    }

    __syncthreads();

    if (threadIdx.x == 0) {
        nvshmemx_signal_op(remote_signal, 1,
                           NVSHMEM_SIGNAL_SET,
                           peer_pe);
    }
}
```

This example illustrates the programming idea, although real NVSHMEM code must carefully allocate symmetric memory and use valid NVSHMEM device APIs. The key point is that a GPU kernel can perform a remote update and then signal another PE.

The synchronization issue is crucial. Communication and synchronization are separate in one-sided models. A put or remote write does not automatically mean the remote side can safely consume the data.

A safer conceptual pattern is:

producer writes data → producer signals → consumer waits for signal → consumer reads data.

The consumer side may look conceptually like:

```
__global__ void consumer_kernel(float *local_buffer,
                                uint64_t *local_signal,
                                int n) {

    if (threadIdx.x == 0) {
```

```

        nvshmem_signal_wait_until(local_signal,
                                   NVSHMEM_CMP_EQ,
                                   1);
    }

    __syncthreads();

    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    if (tid < n) {
        float x = local_buffer[tid];
        local_buffer[tid] = x + 1.0f;
    }
}

```

Hardware interpretation:

- Communication can be issued from inside a running GPU kernel.
- The CPU does not need to regain control between production and communication.
- Fine-grained communication can overlap with other thread blocks' computation.
- Incorrect synchronization can cause stale reads, races, or partially visible data.

PyTorch-Level Collective Communication

In deep learning frameworks, GPU communication is often hidden under distributed APIs.

```

import torch
import torch.distributed as dist

def average_gradients(model):
    world_size = dist.get_world_size()

    for parameter in model.parameters():
        if parameter.grad is None:
            continue

        dist.all_reduce(parameter.grad,
                        op=dist.ReduceOp.SUM)
        parameter.grad.div_(world_size)

```

This code may use NCCL underneath when running on NVIDIA GPUs. The programmer sees a high-level all-reduce, while the backend chooses topology-aware collective algorithms.

The communication cost matters because distributed data-parallel training performs gradient synchronization. If the total gradient size is G bytes and there are p GPUs, the all-reduce cost becomes a major scaling bottleneck.

$$T_{\text{step}} = T_{\text{forward}} + T_{\text{backward}} + T_{\text{gradient all-reduce}}.$$

Practical systems attempt to overlap gradient all-reduce with backpropagation by launching communication for earlier layers while later layers are still computing gradients.

User-Level Communication Libraries

CUDA-Aware MPI

MPI was designed long before GPUs became dominant. CUDA-aware MPI extends MPI implementations so that they can recognize GPU buffers and use GPU-aware transports.

Key ideas:

- API calls are host-side.
- MPI can distinguish host and device pointers using mechanisms such as unified virtual addressing.
- For off-node transfers, GPUDirect RDMA may be used.
- For intra-node transfers, CUDA IPC may be used.
- MPI is portable and deeply used in HPC.

Strengths:

- mature ecosystem;
- scalable to very large systems;
- portable across machines and vendors;
- integrates with legacy HPC applications.

Limitations:

- not naturally CUDA stream aware;
- API remains host-side;
- synchronization between CUDA streams and MPI calls must be handled carefully;
- not designed originally around GPU execution.

Unified Virtual Addressing

Unified virtual addressing, or UVA, gives host and device memory a unified virtual address space. This allows software to infer whether a pointer refers to CPU memory or GPU memory.

Before UVA, a copy call needed explicit direction:

host-to-device, device-to-host, device-to-device.

With UVA, pointer attributes can determine the location:

address $\rightarrow$ memory owner.

This is important for CUDA-aware MPI because MPI can inspect a pointer and choose the correct transfer mechanism.

CUDA IPC

CUDA interprocess communication allows one process to share GPU memory with another process through handles. A process can allocate GPU memory, export a handle, and another process can open that handle to access the same memory.

This is especially important for intra-node GPU communication across multiple processes.

Conceptually:

process A allocates GPU memory $\rightarrow$ exports handle $\rightarrow$ process B imports handle $\rightarrow$ process B accesses memory

CUDA IPC avoids host staging and is commonly used by GPU-aware communication libraries.

GDRCopy

GDRCopy is useful for small-message handling. It maps GPU memory into user space so that the CPU can access GPU memory more efficiently for small transfers.

For small messages, latency dominates:

$$T_{\text{small}} \approx \alpha + T_{\text{software overhead}}.$$

Reducing software overhead and avoiding expensive setup can matter more than achieving peak bandwidth.

UCX

UCX is a communication framework that provides abstraction over multiple transports. Higher-level libraries such as MPI, OpenSHMEM-like systems, or sometimes other communication frameworks can use UCX underneath.

UCX can select among transports such as:

- TCP;
- InfiniBand RDMA;
- GPUDirect RDMA;
- CUDA IPC;
- GDRCopy;
- shared memory transports.

The lecture's UCTrace tool intercepts UCX calls to visualize which transport was used and how much communication occurred through each path.

NCCL and RCCL

NCCL is NVIDIA's GPU collective communication library. RCCL is AMD's corresponding library with a similar API style.

NCCL is commonly used in deep learning for:

- all-reduce;
- all-gather;
- reduce-scatter;
- broadcast;
- point-to-point send and receive.

NCCL is optimized for NVIDIA GPU topologies including PCIe, NVLink, NVSwitch, and InfiniBand. Its key advantage is that it is CUDA stream aware.

A typical collective can be modeled as:

application $\rightarrow$ NCCL API $\rightarrow$ topology-aware algorithm $\rightarrow$ GPU kernels and transports.

NCCL often chooses algorithms based on message size and topology. For example, ring algorithms may be favorable for bandwidth-heavy large messages, while tree-like algorithms can reduce latency for smaller messages.

NVSHMEM

NVSHMEM implements a GPU-oriented partitioned global address space model. Each PE has local private memory and symmetric memory that can be accessed remotely.

Key properties:

- supports host-side and device-side APIs;
- supports one-sided put and get operations;
- supports point-to-point and collective operations;
- supports CUDA streams;
- can expose fine-grained communication from inside kernels;
- separates communication from synchronization.

The biggest conceptual shift is that communication can look like memory access. This is powerful but dangerous. The programmer must reason carefully about memory ordering, visibility, and synchronization.

Synchronization Semantics

Two-Sided Communication

MPI and NCCL are primarily two-sided or collective communication systems. A send generally matches a receive, and a collective involves all participating ranks or communicators.

Synchronization is often implicit in the communication semantics, though not necessarily in the CUDA stream semantics.

For MPI:

CUDA stream work $\nRightarrow$ MPI automatically knows data is ready.

For NCCL:

stream order $\Rightarrow$ communication waits for previous stream work.

One-Sided Communication

NVSHMEM-style put and get operations separate communication and synchronization.

A remote write is not the same as a remote agreement that the data is ready. The producer and consumer need an explicit protocol.

A safe producer-consumer protocol is:

1. producer writes payload;
2. producer enforces required ordering;
3. producer writes or signals readiness flag;
4. consumer waits until readiness flag is visible;
5. consumer reads payload.

The logical correctness condition is:

consumer reads payload only after producer payload write is visible.

If this condition is violated, the consumer may observe stale or partially updated data.

Performance Intuition

Why Host Involvement Hurts

CPU involvement can hurt for several reasons:

- host API calls add launch and orchestration overhead;
- CPU-side synchronization can force kernels to complete before communication starts;
- staging through host memory adds extra copies;
- CPU proxy threads can become bottlenecks;
- host involvement increases latency for fine-grained communication.

For large bulk communication, bandwidth and topology dominate. For fine-grained communication, latency and control overhead dominate.

Why Direct Data Paths Matter

Every unnecessary copy consumes bandwidth and latency. If a message is staged through host memory, the system moves the same payload multiple times.

For n bytes:

one direct transfer = n bytes moved across the critical link,

while staging can require:

staged transfer $\geq 2n$ bytes moved.

The staged version also consumes CPU memory bandwidth and may require synchronization.

Why Stream Awareness Matters

CUDA streams define an ordering of GPU work. If a communication library is stream aware, communication can naturally depend on previous GPU computation without a full device synchronization.

For stream-aware communication:

$$K_1 \rightarrow C_1 \rightarrow K_2$$

can be enqueued into the same stream, where K_1 and K_2 are kernels and C_1 is communication.

For non-stream-aware communication, the programmer may need:

$$K_1 \rightarrow \text{cudaStreamSynchronize} \rightarrow C_1 \rightarrow K_2.$$

The synchronization may prevent useful overlap.

Why Fine-Grained Device Communication Helps

Fine-grained communication is useful when data becomes ready incrementally. Instead of waiting for an entire kernel to finish, a block or warp can communicate a tile, boundary region, or sparse update as soon as it is ready.

This can reduce exposed latency:

$$T_{\text{exposed}} = T_{\text{comm}} - T_{\text{overlapped}}.$$

The goal is to maximize $T_{\text{overlapped}}$.

However, device-side communication is not free. It may:

- consume warp slots;
- create polling overhead;
- increase register pressure;
- complicate memory consistency;
- make performance harder to profile.

Why Performance Numbers Are Hard to Generalize

The lecture deliberately avoids universal performance claims because the best library and communication mode depend on:

- message size;
- communication frequency;
- intra-node versus inter-node traffic;
- PCIe, NVLink, or NVSwitch topology;
- NIC locality;
- GPU architecture;
- library version;
- driver version;
- UCX configuration;
- application communication pattern;
- synchronization requirements.

A benchmark result without topology and configuration details is incomplete.

Profiling and Visualization Tools

Why Profiling GPU Communication Is Difficult

Some communication is visible as explicit API calls. Other communication appears as memory access. For example, peer-to-peer memory loads and stores may not appear clearly as communication in ordinary profiling views.

This makes it difficult to answer:

- which GPU communicated with which other GPU;
- how many bytes moved;
- which transport was used;
- whether CUDA IPC, GDRCopy, or RDMA was selected;
- which source-code line caused communication;
- whether the observed behavior matches expected topology.

Snoopy

The lecture describes Snoopy as a tool for visualizing peer-to-peer GPU communication. It can help identify communication caused by remote memory accesses that may otherwise appear as ordinary memory operations.

A key use case is source attribution: identifying that a particular line of code causes a significant fraction of remote data movement.

UCTrace

UCTrace intercepts UCX calls and visualizes communication and transport usage. It can show:

- communication graph among GPUs or ranks;
- communication matrix;
- transport layer selection;
- percentage of communication through CUDA IPC, GDRCopy, or network transports;
- differences between MPI implementations or configurations.

This is useful for different roles:

- **application programmers**: identify communication bottlenecks;
- **system administrators**: verify that the expected transports are enabled;
- **library developers**: verify protocol and algorithm behavior;
- **performance engineers**: connect topology to measured communication.

Visual Concept

Draw a communication matrix where rows are source GPUs and columns are destination GPUs. Color entries by bytes transferred. Next to it, draw a graph where GPUs are nodes and edges are labeled with transports such as CUDA IPC, GDRCopy, or RDMA.

Comparison of MPI, NCCL, and NVSHMEM

Programming Model Comparison

Feature	CUDA-Aware MPI	NCCL or RCCL	NVSHMEM
Primary model	Message passing	Collectives and GPU communication	Partitioned global address space
API location	Host-side	Host-side currently, device-side APIs discussed for future systems	Host-side and device-side
Stream awareness	Limited or absent in standard MPI	Native CUDA stream support	Native stream support
Communication style	Mostly two-sided send and receive, plus collectives	Optimized collectives and point-to-point	One-sided put and get, plus collectives
Synchronization	Often tied to communication calls	Stream ordered and collective semantics	Explicit signals, waits, and memory-order reasoning
Best fit	Portable HPC applications	Deep learning collectives and topology-aware GPU communication	Fine-grained GPU-initiated communication
Main risk	Extra synchronization and host control	Less general programming model than MPI	Race conditions and debugging complexity

Migration Difficulty

The lecture emphasizes that switching among these libraries is not trivial. Moving from NCCL to NVSHMEM, for example, is not just replacing function names. It often requires restructuring the program.

This is because the programming model changes:

two-sided communication $\neq$ one-sided remote memory access.

A send-receive program often assumes matching communication operations. A one-sided program must explicitly manage ownership, visibility, and synchronization.

Unicorn: A Uniform API Research Direction

The lecture briefly describes a research project called Unicorn. The goal is to provide a uniform API over multiple communication backends while preserving stream awareness and avoiding significant overhead.

The desired abstraction is:

application $\rightarrow$ uniform communication API $\rightarrow$ {MPI, NCCL, NVSHMEM, ...}.

The research challenge is that different libraries have genuinely different semantics. A good abstraction must avoid hiding important correctness and performance differences.

Common Misconceptions

Misconception: GPU Communication Is Only About Bandwidth

Bandwidth matters, but communication performance also depends on latency, synchronization, control overhead, topology, routing, and software stack behavior.

For small messages:

$$T_{\text{comm}} \approx \alpha + T_{\text{control}}.$$

For large messages:

$$T_{\text{comm}} \approx \frac{n}{B}.$$

A bandwidth-only view misses small-message and fine-grained communication bottlenecks.

Misconception: GPUDirect RDMA Means the CPU Is Gone

GPUDirect RDMA removes the CPU from the data path, but not necessarily from the control path. The CPU may still register memory, post operations, maintain proxy threads, or coordinate synchronization.

A more precise statement is:

GPUDirect RDMA improves data path $\nrightarrow$ fully device-native control.

Misconception: Device-Side Communication Is Always Faster

Device-side communication can reduce latency and improve overlap, but it can also consume GPU resources and complicate synchronization. It is most useful when the application has fine-grained or irregular communication that benefits from intrakernel communication.

Misconception: NCCL and MPI Are Interchangeable

Both can move data among ranks, but they are optimized for different programming models. MPI is general and portable. NCCL is deeply optimized for GPU collectives and CUDA stream integration.

Misconception: One-Sided Communication Avoids Synchronization

One-sided communication separates communication from synchronization. It does not eliminate synchronization. In fact, the programmer must often reason more explicitly about ordering.

Misconception: Profilers Always Show Communication Clearly

Remote memory access may appear as ordinary memory access. Without specialized tracing, it can be difficult to determine whether a load or store caused peer-to-peer communication.

Practical Takeaways

For GPU Programmers

- Always identify whether your communication is intra-node or inter-node.
- Know whether your data path stages through host memory.
- Check whether peer-to-peer access is enabled.
- Use topology-aware libraries such as NCCL when doing deep learning collectives.
- Avoid unnecessary `cudaDeviceSynchronize` or `cudaStreamSynchronize` calls.
- Prefer stream-aware communication when possible.
- Use NVSHMEM-like models only when you are prepared to reason carefully about synchronization.

For Performance Engineers

- Measure message size distributions, not only total bytes.
- Separate startup latency from bandwidth cost.
- Inspect transport selection through UCX or communication tracing tools.
- Verify whether CUDA IPC, GDRCopy, GPUDirect RDMA, NVLink, or NVSwitch is actually used.
- Do not trust generic performance claims without topology and configuration details.

For Deep Learning Systems

Distributed training performance depends heavily on collective communication. Gradient synchronization, tensor parallelism, sequence parallelism, expert parallelism, and pipeline scheduling all rely on efficient communication.

A useful training-step model is:

$$T_{\text{iteration}} = T_{\text{compute}} + T_{\text{memory}} + T_{\text{communication}} - T_{\text{overlap}}.$$

The systems goal is to reduce $T_{\text{communication}}$ and increase T_{overlap} .

For HPC Applications

Legacy MPI applications can benefit from CUDA-aware MPI, but optimal GPU performance may require deeper restructuring:

- use GPU-resident data structures;
- avoid unnecessary host transfers;
- overlap halo exchange with computation;
- use GPU-aware transports;
- consider NVSHMEM for irregular fine-grained communication.

Project or Experiment Ideas

Experiment 1: Host-Staged Versus Peer-to-Peer Copies

Implement a benchmark comparing:

1. GPU to CPU to GPU copy;
2. direct GPU peer-to-peer copy;
3. NCCL point-to-point send and receive if available.

Measure:

$$T(n)$$

for different message sizes n . Plot latency-dominated and bandwidth-dominated regimes.

Experiment 2: MPI Versus NCCL Stream Behavior

Write a kernel that fills a GPU buffer, then communicate the buffer using CUDA-aware MPI and NCCL.

Compare:

- MPI with explicit stream synchronization;
- NCCL enqueued into the same stream;
- NCCL enqueued into a separate stream with events.

Measure exposed synchronization overhead.

Experiment 3: Ring All-Reduce Scaling

Implement or benchmark all-reduce across p GPUs and compare measured time to:

$$T_{\text{ring}} \approx 2(p-1)\alpha + 2\frac{p-1}{p} \frac{n}{B}.$$

Vary p and n . Observe the transition from latency-bound to bandwidth-bound behavior.

Experiment 4: NVSHMEM Producer-Consumer Protocol

Write a small NVSHMEM program where one GPU produces data and signals another GPU. Intentionally remove the signal or wait and observe incorrect behavior.

The goal is to internalize:

$$\text{communication} \neq \text{synchronization}.$$

Experiment 5: Transport Inspection with UCX

Run a CUDA-aware MPI program with different UCX settings. Try to observe whether the program uses CUDA IPC, GDRCopy, or GPUDirect RDMA.

Record:

- message size;
- transport selected;
- latency;
- bandwidth;
- topology.

Experiment 6: Topology-Aware Deep Learning Collective Benchmark

Use PyTorch distributed with NCCL and benchmark all-reduce for different tensor sizes. Compare:

- GPUs connected through NVLink;
- GPUs communicating over PCIe;
- multi-node GPUs over InfiniBand if available.

Measure how topology changes the effective bandwidth.

Final Mental Model

Modern GPU systems are no longer CPU-centered machines with passive accelerators. They are distributed systems of GPUs, switches, NICs, memory hierarchies, and software protocols. The hard problem is not only computing faster, but moving data at the right time, through the right path, with the least unnecessary synchronization.

The communication stack can be understood through four questions:

Who calls the API?

Where do the bytes physically move?

Who registers or prepares the message?

Who triggers the transfer?

The more answers move from CPU to GPU, the more GPU-centric the system becomes. However, GPU-centric communication is not automatically better. It trades CPU overhead for greater responsibility in synchronization, topology awareness, profiling, and correctness.

The strongest practical mental model is:

Communication performance is a combination of data path, control path, synchronization semantics, topology, and message granularity. A fast multi-GPU program is one that keeps data on GPUs, moves it directly across the best available links, overlaps communication with useful work, and uses the communication library whose semantics match the application.